

```
# Documento técnico
## Sistema de Gestión de Asociados

**Autor:** Carlos Daniel Molina Ordoñez
**Tecnología principal:** C# con .NET 10
**Persistencia:** MySQL
**Tipo de aplicación:** Aplicación de consola
```

---

## ## 1. Descripción general

El Sistema de Gestión de Asociados permite administrar los miembros de una cooperativa y registrar sus movimientos financieros. La aplicación permite registrar, consultar, actualizar y eliminar asociados, además de realizar consignaciones, retiros, consultas de saldo y reportes administrativos.

El sistema aplica reglas de negocio para validar documentos, teléfonos, nombres, montos, saldos disponibles y restricciones de eliminación. La información se conserva en una base de datos MySQL para que permanezca disponible entre ejecuciones de la aplicación.

## ## 2. Estructura del proyecto

El proyecto utiliza una arquitectura por capas para separar la interfaz, las reglas de negocio, el acceso a datos y los modelos del dominio.

```
```text
Sistema gestion de asociados/
├── Data/
│   └── MySqlDatabase.cs
├── Dependencies/
│   └── TrmService.cs
├── Interfaces/
│   ├── IMemberRepository.cs
│   ├── IMemberService.cs
│   ├── IMovementRepository.cs
│   ├── IMovementService.cs
│   └── ITrmService.cs
├── Models/
│   ├── Member.cs
│   ├── MemberMovementSummary.cs
│   ├── Movement.cs
│   ├── MovementType.cs
│   ├── OperationResult.cs
│   └── TrmRate.cs
├── Repositories/
│   ├── MemberRepository.cs
│   └── MovementRepository.cs
├── Services/
│   ├── MemberService.cs
│   └── MovementService.cs
├── UI/
│   └── ConsoleMenu.cs
├── Program.cs
├── README.md
└── Sistema gestion de asociados.csproj
```
```

## ## 3. Capas y módulos

### ### 3.1 Models

Contiene las entidades y tipos principales del dominio:

- ``Member``: información del asociado, como documento, nombres, teléfono, dirección y fecha de registro.
- ``Movement``: consignaciones y retiros asociados a un miembro.
- ``MovementType``: enum que diferencia depósitos y retiros.
- ``MemberMovementSummary``: información agregada para los reportes.
- ``OperationResult``: resultado uniforme para operaciones exitosas o fallidas.
- ``TrmRate``: modelo utilizado para la conversión de saldos a dólares.

### ### 3.2 Interfaces

Define contratos para desacoplar las reglas de negocio de la persistencia y de los servicios externos. Las interfaces principales son ``IMemberRepository``, ``IMovementRepository``, ``IMemberService``, ``IMovementService`` e ``ITrmService``.

### ### 3.3 Repositories

Implementa el acceso a datos mediante MySQL y consultas SQL parametrizadas.

- ``MemberRepository``: operaciones CRUD, búsquedas y validaciones de existencia de asociados.
- ``MovementRepository``: registro, consulta, filtrado por fechas, movimientos más grandes y resúmenes.

### ### 3.4 Services

Contiene las reglas de negocio de la aplicación.

- ``MemberService``: valida y administra asociados, además de calcular saldos y controlar las condiciones de eliminación.
- ``MovementService``: valida consignaciones y retiros, calcula comisiones y evita retiros superiores al saldo disponible.

### ### 3.5 Data

``MySqlDatabase`` centraliza la cadena de conexión, crea conexiones MySQL e inicializa las tablas ``Members`` y ``Movements`` cuando inicia la aplicación.

### ### 3.6 UI

``ConsoleMenu`` implementa la interacción con el usuario mediante un menú de consola. Presenta las opciones, solicita datos y muestra resultados sin contener la lógica principal del negocio.

### ### 3.7 Dependencies

``TrmService`` consume el servicio externo de TRM para convertir saldos de pesos colombianos a dólares como valor de referencia.

## ## 4. Tecnologías utilizadas

- C#.
- .NET 10.
- MySQL.
- Paquete ``MySqlConnection`` para acceso ADO.NET a MySQL.
- SQL parametrizado.
- LINQ para consultas y transformaciones de colecciones.
- ``HttpClient`` y JSON para el consumo de la TRM.
- Git y GitHub para control de versiones.
- Aplicación de consola como interfaz de usuario.

## ## 5. Flujo general del sistema

1. ``Program.cs`` configura la cultura regional ``es-CO``.

2. Se crea `ConsoleMenu`.
3. `MySQLDatabase.Initialize()` abre la conexión y crea las tablas si aún no existen.
4. Se instancian los repositorios MySQL.
5. Los servicios reciben los repositorios mediante sus constructores.
6. El menú presenta las operaciones disponibles.
7. El usuario selecciona una acción y proporciona los datos solicitados.
8. El servicio valida los datos y aplica las reglas de negocio.
9. El repositorio ejecuta las consultas parametrizadas contra MySQL.
10. El resultado se devuelve a la interfaz y se muestra al usuario.

## ## 6. Servicios y reglas principales

### ### Registro de asociados

- El documento es obligatorio y debe contener únicamente números.
- Los nombres y apellidos no pueden contener números.
- El teléfono es opcional, pero no puede repetirse cuando se informa.
- El documento no puede repetirse.
- La dirección es obligatoria.

### ### Movimientos financieros

- Las consignaciones y retiros deben tener un valor mayor que cero.
- El asociado debe existir antes de registrar un movimiento.
- Cada movimiento recibe un identificador de transacción.
- Los retiros superiores a 1.000.000 COP aplican una comisión de 8.000 COP.
- No se permite retirar un valor superior al saldo disponible.
- El saldo se calcula a partir de los movimientos, no se almacena como dato duplicado.

### ### Eliminación

Un asociado no puede eliminarse si tiene movimientos registrados o un saldo diferente de cero.

### ### Reportes

El sistema ofrece consultas de movimientos por asociado y por rango de fechas, los diez movimientos más grandes, saldos en COP, conversión a USD y resúmenes de actividad.

## ## 7. Persistencia MySQL

La aplicación utiliza dos tablas relacionadas:

- `Members`: almacena la información personal y la fecha de registro.
- `Movements`: almacena las transacciones y referencia al asociado mediante `MemberId`.

La relación entre las tablas utiliza una clave foránea. También existen restricciones únicas para evitar documentos, teléfonos informados e identificadores de transacción duplicados.

La conexión se configura mediante la variable de entorno  
`MYSQL\_CONNECTION\_STRING`:

```
```bash
export
MYSQL_CONNECTION_STRING="Server=127.0.0.1;Port=3306;Database=cooperative_managem
ent;User ID=root;Password=1234;"
```
```

La base de datos debe existir antes de iniciar la aplicación:

```
```sql
CREATE DATABASE IF NOT EXISTS cooperative_management;
```
```

## ## 8. Fragmentos de código relevantes

### ### Inicialización de la base de datos

```
```csharp
using var connection = CreateConnection();
connection.Open();

using var command = connection.CreateCommand();
command.CommandText = "CREATE TABLE IF NOT EXISTS Members (...)"
command.ExecuteNonQuery();
```
```

### ### Registro mediante consulta parametrizada

```
```csharp
command.CommandText = """
    INSERT INTO Members
        (DocumentNumber, FirstName, LastName, Phone, Address, RegistrationDate)
    VALUES
        (@documentNumber, @firstName, @lastName, @phone, @address,
@registrationDate);
    """;
```
```

### ### Inyección de dependencias por constructor

```
```csharp
public MemberService(
    IMemberRepository memberRepository,
    IMovementRepository movementRepository)
{
    _memberRepository = memberRepository;
    _movementRepository = movementRepository;
}
```
```

## ## 9. Buenas prácticas aplicadas

- Separación de responsabilidades mediante arquitectura por capas.
- Uso de interfaces para reducir el acoplamiento.
- Validación de datos antes de ejecutar operaciones de persistencia.
- Consultas SQL parametrizadas para evitar inyección SQL.
- Restricciones de base de datos para proteger la integridad de la información.
- Reutilización de modelos y resultados de operación.
- Uso de `using` para liberar conexiones, comandos y lectores de datos.
- Configuración de credenciales mediante variables de entorno.
- Control de versiones con Git y publicación en GitHub.
- Uso de nombres descriptivos y métodos pequeños con una responsabilidad clara.

## ## 10. Ejecución

1. Crear la base de datos `cooperative\_management` en MySQL.
2. Configurar `MYSQL\_CONNECTION\_STRING`.
3. Ejecutar el proyecto:

```
```bash
dotnet run
```
```

4. La aplicación crea las tablas automáticamente y muestra el menú principal.